If it is compiled, ***strlen (s)*** is called each time and strlen needs to iterate over the whole of **'s'.** If the compiler is smart enough to work out that strlen is a pure function and that **'s'** is not updated in the loop, then it can remove the redundant extra calls to strlen and make the loop to execute only one time. From these what we can understand, strlen is a pure function because the function takes one variable as a parameter, and accesses it to find its length. This function reads external memory but does not change it, and the value returned derives from the external memory accessed.

Note

Evaluation of pure functions does not cause any side effects to its output

1.4.1 Impure functions

The variables used inside the function may cause side effects though the functions which are not passed with any arguments. In such cases the function is called impure function. When a function depends on variables or functions outside of its definition block, you can never be sure that the function will behave the same every time it's called. For example the mathematical function random() will give different outputs for the same function call.

```
let Random number
let a := random()
    if a > 10 then
       return: a
    else
       return: 10
```

Here the function Random is impure as it is not sure what will be the result when we call the function.

1.4.2 Side-effects (Impure functions)

As you are aware function has side effects when it has observable interaction with the outside world. There are situations our functions can become impure though our goal is to make our functions pure. Just to clarify remember that side effect is not a necessary bad thing.Sometimes they are useful *(especially outside functional programming paradigm)*.

Modify variable outside a function

One of the most popular groups of side effects is modifying the variable outside of function.

For example

```
let y: = 0
(int) inc (int) x:
       y: = y + x;
       return (y)
```

In the above example the value of y get changed inside the function definition due to which the result will change each time. The side effect of the inc () function is it is changing the data of the external visible variable **'y'**. As you can see some side effects are quite easy to spot and some of them may tricky. A good sign that our function impure *(has side effect)* is that it doesn't take any arguments and it doesn't return any value.

From all these examples and definitions what we can understand about the main differences between pure and impure functions are